

Multidimensionale Evaluation LLM-basierter Coding-Agenten

Eine zusätzliche Metrikschicht für generierte Codeartefakte

Chris David Kaufmann, Luca di Siro, Daniel Ertel und Richard Beser

Duale Hochschule Baden-Württemberg Mosbach, Studiengang Wirtschaftsinformatik
chriskaufmann11@gmail.com, luca.di.siro@web.de,
danielertel2002@gmail.com, richardbeser@gmail.com

Zusammenfassung. LLM-basierte Coding-Agenten lösen überschaubare Programmieraufgaben inzwischen fast durchgängig korrekt. Eine rein funktionale Bewertung über bestandene Tests verliert dadurch ihre Trennschärfe. Diese Arbeit entwickelt eine zusätzliche Metrikschicht, die funktionale Korrektheit um Robustheit gegenüber Randfällen, Wartbarkeit, Codeumfang, Constraint Compliance sowie Token- und Laufzeiteffizienz ergänzt. In einem gepaarten Experiment werden Claude Code, OpenAI Codex und Gemini CLI über 288 Läufe verglichen. Die Ergebnisse zeigen einen deutlichen Ceiling-Effekt bei Pass/Fail-Metriken, zugleich jedoch signifikante Unterschiede in Robustheit, Codeumfang, Komplexität, Constraint Compliance und Ressourcenverbrauch. Codex arbeitet am sparsamsten, Claude erzielt bei regulär erzeugten Artefakten die beste Gesamtqualität, Gemini bleibt stabil, benötigt dafür aber ein Vielfaches an Tokens. Damit entsteht eine übertragbare Bewertungsgrundlage für differenziertere Entscheidungen über den betrieblichen Einsatz agentischer Coding-Systeme.

Schlüsselwörter: Coding Agents, Large Language Models, Softwarequalität, Evaluation, Token-Effizienz

Abstract. LLM-based coding agents now solve small programming tasks almost consistently, which deprives functional pass/fail evaluation of its discriminative power. This paper develops an additional metric layer complementing functional correctness with edge-case robustness, maintainability, code size, rule compliance as well as token and runtime efficiency. Comparing Claude Code, OpenAI Codex and Gemini CLI across 288 paired runs, the results confirm a pronounced ceiling effect while revealing significant differences in robustness, code size, complexity, compliance and resource consumption. The work provides a transferable basis for differentiated decisions on the operational use of agentic coding systems.

Keywords: Coding Agents, Large Language Models, Software Quality, Evaluation, Token Efficiency

1 Einleitung

Etablierte Benchmarks bewerten LLM-basierte Codegenerierung überwiegend über das Bestehen vorgegebener Unit-Tests [1]. Das prüft grundlegende Lösbarkeit, verliert aber an Aussagekraft, sobald leistungsfähige Systeme in einem begrenzten Aufgabenraum nahezu identische Pass-Raten erreichen. Dieser Effekt wird als Benchmark-Sättigung beschrieben [2]. Hinzu kommt, dass Testfälle nur einen Ausschnitt der Korrektheit abbilden [3] und nicht-funktionale Merkmale wie Wartbarkeit, Robustheit oder Ressourceneffizienz unberücksichtigt lassen [4].

Die vorliegende Arbeit schließt diese Lücke durch eine zusätzliche Metrikschicht. Sie ersetzt Unit-Tests nicht, sondern ergänzt sie um messbare Qualitäts-, Konformitäts- und Effizienzindikatoren. Der Beitrag besteht aus einer Pipeline, die Codegenerierung, Testausführung, Codeanalyse und Verbrauchsmessung verbindet, aus anbieterübergreifend vergleichbar aufbereiteten Kennzahlen und aus einem gepaarten Experiment, das ihren Mehrwert gegenüber einer reinen Pass/Fail-Bewertung prüft.

Die Untersuchung folgt vier Forschungsfragen. Welchen marginalen Beitrag liefert zusätzlicher Tokenverbrauch zur funktionalen Leistung (F1)? Verbessert ein Self-Refining-Modus die Robustheit gegenüber Randfällen (F2)? Wie beeinflusst die Prompt-Qualität den Ressourcenverbrauch (F3)? Führen präzisere Prompts zugleich zu höherer Gesamtleistung und geringerem Tokenverbrauch (F4)? Diese Fragen werden in H1 bis H4 als prüfbare Hypothesen operationalisiert. Diese Arbeit wurde im Rahmen des Integrationsseminars am Studiengang Wirtschaftsinformatik der DHBW Mosbach unter Leitung von Prof. Dr. Christian Schalles erstellt.

2 Evaluationsframework und Studiendesign

2.1 Experimentaufbau

Die Evaluation erfolgt auf Ebene der agentischen CLI-Systeme und nicht isolierter Basismodelle. Verglichen werden OpenAI Codex in der Spezifikation gpt-4.1 beziehungsweise o3, Anthropic Claude Code mit Claude Sonnet 4.6 und Claude Haiku 4.5 als Hilfsmodell sowie Google Gemini CLI mit gemini-3-flash-preview als Hauptmodell und gemini-3.1-flash-lite als Hilfsmodell. Claude Code und Gemini CLI orchestrieren diese Modelle intern selbst, sodass sich die Befunde auf das jeweilige Gesamtsystem beziehen und nicht allein auf das gewählte Hauptmodell. Alle drei Agenten liefern über die jeweils günstigste kostenpflichtige Zugangsstufe der Anbieter von rund 20 bis 25 Euro monatlich. Der Aufgabenraum umfasst vier Python-Aufgaben, nämlich ISBN-Validierung, FizzBuzz, Entfernen von Duplikaten und Caesar-Chiffre, in je vier Varianten von der Basisfunktion bis zum Umkehrproblem. Variiert werden zudem drei Prompt-Stufen und zwei Architekturmodi. Low steht für vage Anweisungen, Medium für spezifizierte Signaturen und einzelne Randfälle, High ergänzt Rollen, explizite Vorgaben und Prüfhinweise. Direct erzeugt eine Lösung unmittelbar, Self-Refining sieht eine zusätzliche Korrekturschleife vor [5]. Der Modus ist für alle drei Agenten identisch implementiert: Ein einheitlicher Prompt-Zusatz fordert zur Selbstprüfung auf, und bei fehlgeschlagenen sichtbaren

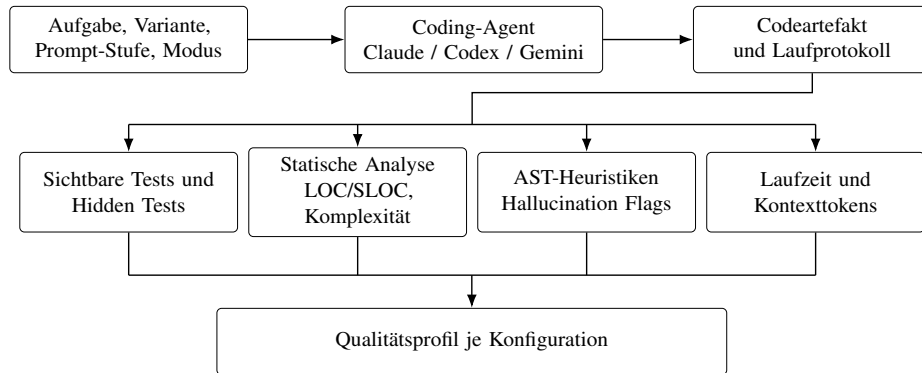


Abbildung 1. Aufbau der Evaluationspipeline.

Tests wird deren Ausgabe als Rückmeldung in eine zweite, letzte Iteration übergeben. Gemessen wird damit eine Reparaturrunde auf Basis sichtbaren Testfeedbacks.

Daraus ergeben sich je Agent 96 Konfigurationen und insgesamt 288 Experimentläufe. Alle drei Agenten bearbeiten dieselben Konfigurationen, sodass jede Aussage auf einem direkten Vergleich bei identischer Aufgabenstellung beruht. Dieses gepaarte Vorgehen hält Aufgabenschwierigkeit, Promptformulierung und Modus konstant, sodass verbleibende Unterschiede dem Agenten zuzurechnen sind. Jeder Lauf erzeugt ein nachvollziehbares Verzeichnis mit Prompt, Agentenausgabe, finaler solution.py, Testresultaten und Messdaten (Abb. 1).

2.2 Operationalisierte Metriken

Die Metrikschicht bildet sechs Perspektiven ab: Funktionalität, Robustheit, Wartbarkeit, Minimalität, Constraint Compliance und Effizienz. Funktionalität wird über Success Rate und Test-Pass-Rate erfasst, ergänzt um verdeckte Tests (Hidden Tests), die dem Agenten nicht bekannt sind und gezielt Randfälle prüfen. Die Wartbarkeit beruht auf drei etablierten Kennzahlen. SLOC zählt die Codezeilen ohne Kommentare und Leerzeilen, die zyklomatische Komplexität die Verzweigungen im Kontrollfluss, und der Maintainability Index fasst beide mit dem Halstead-Volumen zusammen. Sie beantworten, wie aufwendig es wäre, den Code zu verstehen und anzupassen. Hinzu kommen AST-basierte Heuristiken, also regelbasierte Prüfungen auf dem abstrakten Syntaxbaum des erzeugten Codes. Sie erkennen unnötige Bibliotheksimporte, vergessene print-Ausgaben, in den Code gerutschte Text- oder Markdown-Reste und Änderungen an der Testdatei und setzen entsprechende Hallucination Flags. Das ist keine vollständige Halluzinationsmessung, wohl aber eine reproduzierbare Kontrolle typischer Fehlerbilder LLM-generierten Codes [6, 7]. Der Minimality Score bewertet, ob eine Lösung dem Umfang der Aufgabe angemessen bleibt. Er beginnt bei 1,0 und wird vermindert, sobald eine Lösung zu umfangreich ausfällt. Umfasst sie 50 bis 80 Codezeilen, werden 0,2 abgezogen, bei mehr als 80 Codezeilen 0,4. Für die zyklomatische Komplexität gilt dieselbe Staffelung mit den Grenzen 9 bis 12 und darüber. Beide Abzüge können zusammentreffen, der Wert fällt jedoch nicht unter 0. Die Schwellen sind keine externe Norm, sondern eine im

Projekt festgelegte und offengelegte Konvention, die sich am Umfang einer direkten Lösung orientiert. Die Effizienz umfasst Laufzeit, normalisierte Kontexttokens, Cache Ratio und Tokens je bestandenen Testfall.

Ein qualitativer Gesamtscore verdichtet Interface Compliance, Constraint Compliance, Minimality Score und Robustness Score zu einem ungewichteten Mittel. Alle vier liegen zwischen 0 und 1, wobei 1 der bestmögliche Wert ist.

$$Q = \frac{Interface + Constraint + Minimality + Robustness}{4} \quad (1)$$

Die Gleichgewichtung ist bewusst transparent gewählt, da keine empirisch abgesicherte Gewichtung vorliegt. Der Maintainability Index bleibt außen vor, weil er selbst aus Codeumfang und Komplexität berechnet wird und sonst doppelt zählte.

2.3 Token-Normalisierung und statistische Auswertung

Ein direkter Vergleich der Roh-Tokenwerte ist methodisch problematisch, da die Anbieter Eingabe-, Cache-, Ausgabe- und Reasoning-Tokens unterschiedlich ausweisen. Daher werden die Nutzungsdaten auf ein gemeinsames Schema abgebildet. Die technische Vergleichsgröße ist wie folgt definiert:

$$T_{norm} = T_{prompt, inkl. Cache} + T_{completion} + T_{reasoning} \quad (2)$$

Für Claude setzt sich der Promptumfang aus regulären Eingaben sowie neu geschriebenen und gelesenen Cache-Tokens zusammen. Bei Codex sind gecachte Tokens bereits Teil der Eingabe-Tokens und dürfen nicht erneut addiert werden, sonst zählt der Verbrauch doppelt. Bei Gemini werden die Werte über alle an einem Lauf beteiligten Modelle aufsummiert. Ergänzend wird Tokens je bestandenen Testfall berechnet.

Die Zielgrößen sind auf [0,1] begrenzt, teils stark schief verteilt und paarweise erhoben, sodass Verfahren mit Normalverteilungsannahme ausscheiden. Stattdessen kommen nichtparametrische Verfahren zum Einsatz, die statt mit den Messwerten mit deren Rangfolge rechnen und dadurch unempfindlich gegen Ausreißer sind.

Der Friedman-Test prüft, ob sich mehr als zwei verbundene Stichproben unterscheiden, sein Gegenstück für binäre Größen ist der Cochran-Q-Test. Ist ein globaler Test signifikant, folgen paarweise Post-hoc-Vergleiche mit dem Wilcoxon-Test für metrische und dem McNemar-Test für binäre Größen. Da mit deren Zahl die Wahrscheinlichkeit eines Zufallsbefunds steigt, werden alle Post-hoc-Ergebnisse Holm-korrigiert [8, 9].

Jeder Test liefert einen p-Wert. Er gibt an, wie wahrscheinlich ein mindestens so deutliches Ergebnis wäre, wenn zwischen den Gruppen in Wirklichkeit kein Unterschied bestünde. Ein p-Wert von 0,368 besagt, dass eine solche Differenz in gut einem Drittel aller Erhebungen zufällig entstünde und deshalb nichts belegt. Als Entscheidungsschwelle dient ein Signifikanzniveau von $\alpha = 0,05$. Ein nicht signifikantes Ergebnis belegt dabei keine Gleichheit, sondern zeigt nur, dass die Daten für einen Nachweis nicht ausreichen.

H1 wird ergänzend über eine lineare Regression der Test-Pass-Rate auf den logarithmierten Tokenverbrauch geprüft, kontrolliert für Agent und Modus. Der Logarithmus bildet ab, dass jede Verdopplung des Aufwands weniger beiträgt als die vorherige [10].

3 Empirische Evaluation

3.1 Operative Stabilität und faire Artefaktbasis

Bei Codex und Gemini wurden alle 96 Läufe regulär abgeschlossen. Claude erzeugte in 70 Konfigurationen auswertbare Artefakte, 26 Läufe endeten ohne Output, weil das Nutzungslimit des Zugangs erreicht wurde. Die Abbrüche bilden einen geschlossenen Block: Die ersten 70 Läufe verliefen regulär, ab dem 71. brach jeder weitere ab. Sie folgen also nicht der Aufgabenschwierigkeit, sondern dem Zeitpunkt der Ausführung. Deshalb werden zwei Ebenen getrennt: Die Vollausswertung mit $n = 96$ je Agent misst operative Stabilität, die faire Artefaktbasis mit $n = 70$ umfasst nur jene Konfigurationen, für die bei allen drei Agenten echte Codeartefakte vorliegen. Ohne diese Trennung gingen abgebrochene Läufe mit null Tokens und null Laufzeit in die Mittelwerte ein und ließen Claude sparsam erscheinen. Jede Kennzahl ist deshalb nur mit ihrer Basis zu lesen.

In der Vollausswertung schließt Claude 72,9 Prozent aller Läufe erfolgreich ab, Codex 99,0 und Gemini 100,0 Prozent. Der Abstand ist signifikant, geht aber fast vollständig auf die 26 abgebrochenen Läufe zurück. Zwischen Codex und Gemini lässt sich kein Unterschied nachweisen.

Auf der fairen Artefaktbasis verschwindet der funktionale Abstand nahezu vollständig. Die Erfolgsquoten liegen zwischen 98,6 und 100 Prozent, die Anteile bestandener sichtbarer Testfälle sind statistisch nicht zu unterscheiden (Tabelle 1). Genau darin besteht der Ceiling-Effekt. Die klassische Pass/Fail-Bewertung trennt die Agenten in vergleichbaren Läufen nicht mehr.

3.2 Mehrdimensionale Qualitätsunterschiede

Die zusätzlichen Metriken zeichnen ein differenzierteres Bild. Dieselben Agenten, die bei den sichtbaren Tests nicht unterscheidbar waren, fallen bei den verdeckten Randfalltests auseinander. Claude besteht 92,9 Prozent dieser Fälle, Gemini 83,3 und Codex 79,3 Prozent, und dieser Unterschied ist signifikant. Auch Codeumfang und Komplexität trennen die Systeme, wobei Claude die kürzesten Lösungen erzeugt und Codex die am wenigsten verschachtelten. Beim Maintainability Index lässt sich dagegen kein Unterschied nachweisen, weshalb der höhere Wert von Gemini nicht als Qualitätsvorteil gilt. Alle Werte samt Testergebnis führt Tabelle 1.

Am deutlichsten sind die Unterschiede bei der Constraint Compliance. Claude und Codex halten sämtliche Vorgaben ein, Gemini nur in 75,7 Prozent der Läufe. Alle 17 Verstöße gehen auf denselben Fall zurück: Gemini veränderte die vorgegebene Testdatei und ergänzte eigene Testfälle, erkennbar an der gestiegenen Testanzahl. Ein Agent, der das Messinstrument anpasst, an dem er bewertet wird, verletzt eine zentrale Rahmenbedingung. Die Läufe bleiben enthalten, werden über die Constraint Compliance aber sichtbar. Beim Ressourcenverbrauch benötigen Gemini und Claude je bestandener Testfall rund das Doppelte von Codex.

Der qualitative Gesamtscore verdichtet diese Profile. Claude erreicht den höchsten Wert, Gemini den niedrigsten, weil dort zwei der vier Teilwerte schwächer ausfallen. Die paarweisen Post-hoc-Vergleiche bestätigen nach Holm-Korrektur, dass Claude bei

Tabelle 1. Zentrale Ergebnisse in der fairen Artefaktbasis mit $n = 70$ Konfigurationen je Agent

<i>Metrik</i>	<i>Claude</i>	<i>Codex</i>	<i>Gemini</i>	<i>Globaler Test</i>
Success Rate	1,000	0,986	1,000	$p = 0,368$
Test-Pass-Rate	1,000	0,997	1,000	$p = 0,368$
Hidden-Pass-Rate	0,929	0,793	0,833	$p = 0,0135$
Codezeilen (SLOC)	30,50	32,70	37,84	$p < 0,001$
Zyklom. Komplexität	6,48	5,72	6,83	$p < 0,001$
Maintainability Index	48,89	48,41	52,30	$p = 0,741$
Constraint Compliance	1,000	1,000	0,757	$p < 0,001$
Laufzeit (s)	30,23	35,52	51,23	$p < 0,001$
Kontexttokens	112.121	62.801	128.160	$p < 0,001$
Tokens/Testfall	6.998	3.870	7.489	$p < 0,001$
Cache Ratio	0,910	0,830	0,723	$p < 0,001$
Qualitativer Score	0,972	0,942	0,881	$p < 0,001$

den verdeckten Tests vor Codex mit $p = 0,0002$ und vor Gemini mit $p = 0,0084$ liegt, dass Codex einfachere Kontrollflüsse erzeugt als Claude mit $p = 0,0006$ und dass Gemini bei der Constraint Compliance sowie beim Kontexttokenverbrauch schwächer abschneidet, beides mit $p < 0,001$. Je bestandem Testfall ist Gemini nur gegenüber Codex signifikant aufwendiger, gegenüber Claude nicht. Auch die drei Unterschiede im qualitativen Gesamtscore sind einzeln abgesichert. Zwischen Claude und Gemini besteht bei der Komplexität dagegen kein nachweisbarer Unterschied, der globale Effekt geht dort allein auf Codex zurück. Es entsteht damit kein eindimensionales Ranking, sondern ein Profil unterschiedlicher Stärken. Claude liefert die höchste Artefaktqualität, Codex die beste Ressourceneffizienz, und Codex wie Gemini bearbeiten den Aufgabenraum ohne technische Ausfälle.

3.3 Prompt-Qualität, Self-Refining und Grenznutzen

Self-Refining erzeugt keinen belastbaren Robustheitsgewinn. Über 105 gepaarte Läufe je Modus liegt der Anteil bestandener Randfalltests im Self-Refining-Modus mit 84,8 Prozent minimal unter der direkten Generierung mit 85,6 Prozent. Weder dieser Unterschied noch die Unterschiede beim Gesamtscore und beim Tokenverbrauch lassen sich statistisch absichern. H2 wird abgelehnt. Zusätzliche Korrekturschleifen erzeugen also nicht automatisch bessere Artefakte.

Bei der Prompt-Qualität zeigt sich ein anderer Effekt. Über 66 gepaarte Blöcke je Stufe sinkt der Tokenverbrauch mit jeder Stufe, von 107.045 über 101.888 auf 93.262 normalisierte Kontexttokens und damit um rund 13 Prozent. Je bestandem Testfall fällt der Rückgang mit rund 15 Prozent noch etwas stärker aus. Beide Effekte sind signifikant. Funktionale Leistung, Robustheit und Gesamtscore verbessern sich dagegen nicht nachweisbar. H3 wird angenommen, H4 abgelehnt. Präzisere Prompts wirken hier als Effizienzhebel und nicht als Qualitätshebel [11]. Der globale Test weist den Rückgang als gesichert aus, nach Holm-Korrektur bleibt jedoch kein einzelner Vergleich zwischen zwei Stufen signifikant. Der Effekt ist über alle drei Stufen belegt, nicht als stabiler Unterschied zwischen zweien.

Die Regression bestätigt H1 als Sättigungshypothese. Der Koeffizient des logarithmierten Tokenverbrauchs liegt bei $-0,00037$ mit $p = 0,913$ und ist damit nicht von null zu unterscheiden. Ein doppelter Tokeneinsatz verändert die erwartete Test-Pass-Rate

Tabelle 2. Zusammenfassung der Hypothesenprüfung

	<i>Hypothese</i>	<i>Ergebnis</i>	<i>Evidenz</i>
H1	Mehr Tokens erklären keine höhere Test-Pass-Rate.	angenommen	$p = 0,913$, $R^2 = 0,0105$
H2	Self-Refining erhöht die Robustheit.	abgelehnt	$p = 0,179$
H3	Bessere Prompts senken den Tokenverbrauch.	angenommen	$p = 0,0409$
H4	Bessere Prompts erhöhen zugleich die Qualität.	abgelehnt	$p = 0,990$

also praktisch nicht. Das Bestimmtheitsmaß $R^2 = 0,0105$ zeigt, dass das Modell rund ein Prozent der Streuung in der Test-Pass-Rate erklärt. Da die Test-Pass-Rate durch den Ceiling-Effekt kaum streut, wurde die Regression zusätzlich mit der Pass-Rate der verdeckten Tests gerechnet, deren Streuung rund vierzehnmal so groß ist. Auch dort ist der Koeffizient mit $-0,032$ bei $p = 0,476$ nicht von null zu unterscheiden, das Bestimmtheitsmaß liegt bei $0,044$. Der Befund ist damit kein Artefakt der gesättigten Zielgröße. Zusätzlicher Tokenverbrauch liefert im untersuchten Aufgabenraum somit keinen messbaren funktionalen Mehrwert. Tabelle 2 fasst die Prüfung zusammen.

4 Diskussion und Beitrag für die Wirtschaftsinformatik

Die Ergebnisse stützen die Ausgangsthese. Unit-Tests bleiben notwendig, reichen zur Bewertung leistungsfähiger Coding-Agenten aber nicht aus, denn erst die zusätzlichen Metriken legen die Unterschiede offen. Für betriebliche Entscheidungen ist zudem die Trennung von Erhebungsstabilität und Artefaktqualität zentral. Ein universelles Ranking wäre irreführend. Wer Wartungsaufwand minimieren will, achtet auf Robustheit und Codeumfang, wer Betriebskosten minimieren will, auf Tokens je bestandem Testfall.

Auch die Konfiguration hat ökonomische Bedeutung. Prompt-Standards sparen eher Ressourcen, als dass sie Qualität steigern, und zusätzliche Reparaturschleifen sind nicht pauschal gerechtfertigt.

5 Limitationen, Ausblick und Fazit

Die externe Validität ist durch Zahl und Schwierigkeitsgrad der Aufgaben begrenzt, woraus der Ceiling-Effekt folgt. Da die Abbrüche das Ende der Ausführungsreihenfolge trafen, enthält die faire Artefaktbasis keine Läufe der Caesar-Chiffre und deckt drei der vier Aufgaben ab. Für Codex und Gemini war sie gut lösbar, der Ausschluss begünstigt Claude daher nicht. Pro Konfiguration wurde ein Lauf erhoben, stochastische Streuung bleibt unquantifiziert. Die 26 Abbrüche sind eine Beschränkung der Erhebungsumgebung, kein Merkmal des Agenten. Die Hallucination Flags sind Heuristiken, keine Halluzinationsmessung. In 17 Läufen veränderte Gemini die Testdatei, ihre Kennzahlen beruhen daher auf einem veränderten Prüfmaßstab. Der Tokenverbrauch ist zudem eine technische Größe, keine Kostenmetrik. Die Test-Pass-Rate ist wegen des Ceiling-Effekts nur eingeschränkt als Zielgröße geeignet. Künftige Arbeiten sollten stattdessen die Robustheit gegenüber Randfällen heranziehen und komplexere Aufgaben sowie Wiederholungsmessungen einbeziehen. Innerhalb ihres Geltungsbereichs macht die Metrikschicht Unterschiede sichtbar, die Pass/Fail-Benchmarks verdecken.

Literatur

1. Chen, M., Tworek, J., Jun, H., Yuan, Q., et al.: Evaluating large language models trained on code. arXiv:2107.03374 (2021)
2. Akhtar, M., Reuel, A., Soni, P., Ahuja, S., et al.: When AI benchmarks plateau: A systematic study of benchmark saturation. arXiv:2602.16763 (2026)
3. Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In: Advances in Neural Information Processing Systems. vol. 36 (2023)
4. Sun, X., Ståhl, D., Sandahl, K., Kessler, C.: Quality assurance of LLM-generated code: Addressing non-functional quality characteristics. arXiv:2511.10271 (2025)
5. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., et al.: Self-refine: Iterative refinement with self-feedback. arXiv:2303.17651 (2023)
6. Agarwal, V., Pei, Y., Alamir, S., Liu, X.: CodeMirage: Hallucinations in code generated by large language models. arXiv:2408.08333 (2024)
7. Tambon, F., Dakhel, A.M., Nikanjam, A., Khomh, F., Desmarais, M.C., Antoniol, G.: Bugs in large language models generated code: An empirical study. arXiv:2403.08937 (2024)
8. Friedman, M.: The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association* 32(200), 675–701 (1937)
9. Holm, S.: A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6(2), 65–70 (1979)
10. Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., et al.: Scaling laws for neural language models. arXiv:2001.08361 (2020)
11. Sahoo, P., Singh, A.K., Saha, S., Jain, V., Mondal, S., Chadha, A.: A systematic survey of prompt engineering in large language models: Techniques and applications. arXiv:2402.07927 (2025)